

VOLODYMYR STETSENKO

Independent Security Researcher

Ethena

sUSDe Cross-chain Controls & Minting Interfaces Security Review

Original research: 22 May 2026

Report date: 8 October 2026

Independent Security Research Report

Contents

Executive Summary 2

Scope & System Overview 3

Source & State Verification 4

Methodology & Validation 5

I-01 | Independent Restriction State 6

Cross-chain Evidence & Impact 7

I-02 | Nonce Domain Truncation 8

I-03 | Signed-order Schema Consistency 9

Conclusions & Recommendations 10

References & Reproduction Notes 11

0	0	0	0	3
Critical	High	Medium	Low	Informational

Executive Summary

This independent assessment examines restriction enforcement between Ethereum and Mantle sUSDe contracts and the nonce and signing interfaces of EthenaMinting. It combines deployment-source verification, historical fork tests, pinned on-chain queries, and review of the protocol's published signing client.

Three Informational findings are documented. The reviewed behaviors were confirmed by the documented checks, but the evidence does not establish unauthorized asset creation, theft from unrelated users, or a material production denial of service. No Critical, High, Medium, or Low finding is assigned within this focused scope.

Findings at a glance

ID	Finding	Severity
I-01	Cross-chain restriction state is administered independently	Informational
I-02	Nonce indexing uses only the low 64 bits	Informational
I-03	Signed-order field widths differ from the Solidity structure	Informational

The cross-chain scenario demonstrates that a holder restricted only on Ethereum can move an existing, locally unrestricted Mantle balance to a fresh address and redeem corresponding backing through the modeled bridge application path. Local blacklist enforcement works. A documented requirement for globally synchronized restrictions was not established, and no directly affected restricted holder was identified at either sampled state.

Nonce aliasing can reject distinct high-valued nonces belonging to the same benefactor. The official example client uses timestamp-based nonces within the effective 64-bit domain. Its typed-data schema also matches the contract's literal schema, avoiding the integration mismatch described in I-03.

Evidence summary

- Four deployments recompiled and compared with historical and current runtime code, excluding compiler-declared immutable fields.
- Seven historical fork-suite tests, seven local control tests, and three focused decode-order tests passed.
- All 574 historical and 958 current Ethereum restricted accounts checked on Mantle; all direct balances were zero.
- Reciprocal peers and route rate limits confirmed at both pinned snapshots.

Recommendations focus on explicit restriction policy, documented nonce bounds, and a canonical signing schema. No remediation supplied by the protocol team was assessed.

Scope & System Overview

Deployment scope

Network	Component	Address
Ethereum	StakedUSDeV2	`0x9D39A5DE30e57443BfF2A8307A4256c8797A3497`
Ethereum	StakedUSDeOFTAdapter	`0x211Cc4DD073734dA055fbF44a2b4667d5E5fE5d2`
Mantle	StakedUSDeOFT	`0x211Cc4DD073734dA055fbF44a2b4667d5E5fE5d2`
Ethereum	EthenaMinting	`0xe3490297a08d6fC8Da46Edb7B6142E4F461b62D3`

Identical addresses on different networks identify separate contracts and storage. Ethereum chain ID is 1; Mantle chain ID is 5000. LayerZero endpoint identifiers are 30101 and 30181 respectively.

Relevant asset and control flows

Mantle to Ethereum: StakedUSDeOFT checks the outbound route limit, burns the sender's representation, and submits a LayerZero message. Upon delivery, the Ethereum adapter releases corresponding sUSDe backing. Its recipient-role check redirects credit to the adapter owner when the recipient is fully restricted.

Local restrictions: Ethereum sUSDe uses `FULL_RESTRICTED_STAKER_ROLE`; Mantle uses the OFT's `blackList` mapping. The owner or designated blacklister can update the Mantle list. The inherited burn path reaches the local blacklist check.

Minting orders: EthenaMinting hashes an order, validates the benefactor's signature or an accepted delegated signer, checks authorization and order conditions, and deduplicates the benefactor's nonce. This review targets bitmap indexing and schema consistency, rather than the complete mint/redemption lifecycle.

Assessment boundary

Reviewed paths include restriction administration, transfer/burn/credit inheritance, route rate limiting, order encoding, signature prerequisites, and nonce accounting. Dependencies were traced where needed to interpret those paths.

Protocol-wide economic backing, custody, governance, other networks, LayerZero DVN/executor security, production operator procedures, and the full minting service are outside scope. This is independent research, not a commissioned protocol-wide audit.

Source & State Verification

Explorer-verified source bundles were fetched for all four contracts. The original local minter, adapter, and Mantle bundles matched all 25, 40, and 42 source files respectively. StakedUSDeV2 was reviewed from its explorer bundle. Source references below use filenames and line numbers within these bundles. [R1-R4]

Recompilation

Contract	Solidity	Optimizer runs / EVM
EthenaMinting	0.8.20	20,000 / Shanghai
StakedUSDeOFTAdapter	0.8.22	200 / Paris
StakedUSDeOFT	0.8.22	200 / Paris
StakedUSDeV2	0.8.19	20,000 / Paris

Compiled runtime code, including metadata, matched the deployed runtime after masking only compiler-declared immutable slots. Adapter and OFT compilation used `metadata.useLiteralContent=true`. Deployment-specific immutable values were excluded from this comparison; this is not an unqualified byte-for-byte deployment reproduction. The observed runtime bytes were unchanged between the two sampled blocks for each contract.

Pinned snapshots

Snapshot	Ethereum block / UTC	Mantle block / UTC
Historical	25,151,180 / 22 May 2026, 13:44:47	95,663,600 / 22 May 2026, 13:45:12
Follow-up	26,140,479 / 7 Oct 2026, 12:22:23	101,622,716 / 7 Oct 2026, 12:22:24

Historical Ethereum block hash:
`0x319c9af3e3962e3d52c20f3ab92860831acd32427f9fcff9cbcd3a3d1500f2a9`

Historical Mantle block hash:
`0xedea998c707e6cc6e977c56c7df5683e834845223e9b6b2a8bdf02fbd5c571f0`

Follow-up Ethereum block hash:
`0x2df77ef2ebe554b49cd7dafdb9945894b29192700d04bc2143405fb835788a12`

Follow-up Mantle block hash:
`0x4986b0f2f46a7ad48f15ff1d1cea5096b13c41ca78058ad34975130d388cd8e7`

The companion `Evidence-Summary.json` records runtime hashes, compiler settings, source comparisons, and snapshot identifiers. The source identity is the verified deployment bundle; no repository commit is substituted for an unverified deployment association.

Methodology & Validation

Manual review traced inherited enforcement and accounting paths. Read-only RPC queries were pinned to the blocks identified above. Active Ethereum restrictions were reconstructed from ordered role grant/revocation events and individually confirmed with `hasRole`; corresponding Mantle balances and blacklist states were then queried. Failed RPC responses were retried and excluded from conclusions until resolved.

Executed checks

Test group	Result	Evidence boundary
Historical bridge fork tests	4 passed	Configuration, L1 restriction, recipient redirection, modeled return flow
Nonce tests	3 passed	Arithmetic, reference consumption, deployed-bytecode rejection after seeded bitmap state
Isolated local controls	7 passed	Debit, transfer, credit, owner authorization, nonce arithmetic
Decode-order checks	3 passed	Branch-selection model: valid true/false returns and an empty-data revert

The decode-order checks ran on 8 October 2026; the other tests and chain queries ran on 7 October. Foundry 1.7.1 and Solidity 0.8.23 compiled the test harnesses. The original nonce harness's storage-slot search was replaced with a recorded-access assertion for bitmap slot 9; the fork was explicitly pinned. These test compiler settings are separate from the deployed-source recompilation settings.

The bridge test imposed an Ethereum restriction through impersonation of the authorized manager. It used an existing Mantle holder balance, executed a source `send`, and delivered a reconstructed OFT payload by impersonating the destination endpoint. It therefore validates the application behavior under the stated conditions, not DVN verification, permissionless message injection, or complete authenticated cross-chain transport.

The nonce fork test seeded a consumed bitmap bit using a test cheatcode. It demonstrates deployed-bytecode behavior for that state, not an attacker capability to write contract storage or a production-signed order execution.

Risk classification

Critical: catastrophic compromise or substantial direct asset loss. **High:** significant exploitable security impact. **Medium:** material impact constrained by prerequisites. **Low:** limited demonstrated security impact. **Informational:** a design, documentation, or integration concern without demonstrated material exploitation in the assessed configuration.

Ratings consider impact, prerequisites, observed exposure, and existing controls. They are independent assessments rather than bounty determinations. The `I` prefix identifies Informational findings; numbers distinguish individual observations. All three behaviors remain present in the sampled code; no team-confirmed remediation status is inferred.

I-01 | Independent Restriction State

Severity: Informational
Category: Cross-chain policy consistency
Status: Behavior verified; remediation not assessed
Locations: StakedUSDeOFT.sol, lines 56-59 and 90-94; StakedUSDeOFTAdapter.sol, lines 38-55

Description

Mantle restrictions are stored locally, while the Ethereum adapter checks the destination recipient's role in the underlying token. No atomic propagation of an Ethereum restriction into the Mantle mapping is implemented in these contracts. A holder restricted on Ethereum can remain transferable on Mantle until an authorized local update occurs. [R1-R3]

Mantle: StakedUSDeOFT.sol, lines 56-59

```
56      function updateBlackList(address _user, bool _isBlackListed) external {
57          if (msg.sender != blackLister && msg.sender != owner()) revert OnlyBlackLister();
58          blackList[_user] = _isBlackListed;
59          emit BlackListUpdated(_user, _isBlackListed);
```

Ethereum: StakedUSDeOFTAdapter.sol, lines 44-54

```
44          (bool success, bytes memory data) = address(innerToken).call(
45              abi.encodeWithSignature("hasRole(bytes32,address)", FULL_RESTRICTED_STAKER_ROLE, _to)
46          );
47          bool isBlackListed = abi.decode(data, (bool));
48
49          // If the call fails, OR recipient is blacklisted, emit an event, redistribute funds, and credit
        the owner
50          if (!success || isBlackListed) {
51              emit RedistributeFunds(_to, _amountLD);
52              return super._credit(owner(), _amountLD, _srcEid);
53          } else {
54              return super._credit(_to, _amountLD, _srcEid);
```

If that holder already owns Mantle tokens and moves them to an unrestricted address, the destination adapter checks the chosen recipient. Successful credit releases backing corresponding to the source burn; it does not establish unbacked issuance or theft of another holder's assets.

Existing enforcement

The Mantle `_update` override checks both sender and recipient. Outbound `_debit` calls the inherited burn, which invokes `_update`. A locally blacklisted holder cannot bypass this by using `send`. The owner remains authorized to update the blacklist when `blackLister` is zero. Adding a redundant outbound check would not solve cross-network state divergence.

Cross-chain Evidence & Impact

Observed exposure

Measurement	Historical snapshot	Follow-up snapshot
Active Ethereum full restrictions	574	958
Confirmed with hasRole	574 of 574	958 of 958
Restricted accounts with positive direct Mantle balance	0	0
Ethereum-restricted accounts also blacklisted on Mantle	0 of 574	0 of 958
Unresolved account-query errors	0	0

The denominator is the entire confirmed Ethereum-restricted set, not only positive-balance holders. None of those addresses was locally blacklisted on Mantle at either snapshot. This shows non-equivalence of these two lists at those times, not the absence of every operational synchronization process.

This is a direct address intersection at two paired snapshots, not a continuous transaction history or attribution of beneficial ownership through intermediaries. It does not establish that the scenario could never arise. It does establish that these sampled restricted addresses held no direct Mantle position to move.

Fork result and route limits

The historical fixture held 2,790.308410791785234128 Mantle sUSDe before the test imposed an Ethereum restriction. The modeled return credited 2,790.308410 sUSDe to a fresh Ethereum recipient; source dust remained below the shared-decimal unit. A control delivery to a restricted recipient redirected backing to the adapter owner. The fixture holder was not an observed restricted holder in the original chain state.

Both sampled routes had reciprocal peers and a rate-limit capacity of 10,000,000 sUSDe with a 3,600-second decay window. Read-only capacity queries returned full available capacity at the sampled blocks. Mantle outbound `_debit` enforces this limit; the Ethereum inbound credit does not independently apply the reverse route's outbound limit. The limiter replenishes linearly and is not a strict rolling-hour volume quota. [R2, R3]

Impact and recommendation

A restriction inconsistency could matter if policy requires holder-wide enforcement across networks. Neither that requirement nor a materially affected holder was established here, so the finding is Informational. Adapter reserves and aggregate bridged supply are not demonstrated funds at risk.

Define the intended cross-chain restriction invariant, responsible operators, propagation latency, and treatment of in-flight transfers. Where consistency is required, reconcile the existing authorized lists and monitor divergent states. Any automated propagation should authenticate its source and handle retries, duplicates, ordering, and revocation. Asynchronous updates cannot guarantee simultaneous enforcement on all chains.

I-02 | Nonce Domain Truncation

Severity: Informational
Category: Input-domain consistency
Status: Behavior verified; remediation not assessed
Locations: EthenaMinting.sol, lines 560-568 and 604-606

Description

The public interface accepts a 128-bit nonce, but slot selection truncates it to 64 bits. The low eight bits select a bit within that slot. Consequently, different nonces with identical low 64 bits share one consumed-order entry for the same sender. [R4]

EthenaMinting.sol, lines 560-568

```
560   function verifyNonce(address sender, uint128 nonce) public view override returns (uint128, uint256,
        uint256) {
561     if (nonce == 0) revert InvalidNonce();
562     uint128 invalidatorSlot = uint64(nonce) >> 8;
563     uint256 invalidatorBit = 1 << uint8(nonce);
564     uint256 invalidator = _orderBitmaps[sender][invalidatorSlot];
565     if (invalidator & invalidatorBit != 0) revert InvalidNonce();
566
567     return (invalidatorSlot, invalidator, invalidatorBit);
568   }
```

For example, `1` and `18446744073709551617` both map to slot 0, bit mask 2. After that bit is consumed, both are rejected. The reference test exercised consumption; the fork test confirmed rejection in deployed bytecode after seeding the same bitmap state. A non-colliding nonce remained accepted by `verifyNonce`.

Impact and integration context

The bitmap is sender-specific. Order verification requires the benefactor's valid signature, an accepted delegated signer, or the applicable EIP-1271 authorization, alongside other order checks. The official example client chooses the nonce: its TypeScript implementation uses the expiry timestamp, and the Python implementation uses the current Unix timestamp plus 60 seconds. Those example values fit the effective domain. A malicious third party causing another benefactor's collision was not demonstrated. [R4, R5]

This is false-duplicate rejection rather than a replay bypass. This assessment verifies the specified deployment and its integration assumptions; it does not claim novelty for the nonce-truncation pattern.

Recommendation

Document and enforce the supported nonce domain, or design a compatible full-width migration. A changed slot derivation must preserve replay protection for already-consumed orders and outstanding signatures. Test boundary values, colliding pairs, and the actual signing integration before changing the live interface.

I-03 | Signed-order Schema Consistency

Severity: Informational
Category: Signing / integration consistency
Status: Schema mismatch verified; remediation not assessed
Locations: IEthenaMinting.sol, lines 45-55; EthenaMinting.sol, lines 36-38 and 480-493

Description

The Solidity structure declares a 120-bit expiry and a 128-bit nonce. The literal EIP-712 order type reverses those widths. These are different type hashes, even when the values fit both domains. [R4]

IEthenaMinting.sol, lines 45-55

```
45  struct Order {
46    string order_id;
47    OrderType order_type;
48    uint120 expiry;
49    uint128 nonce;
50    address benefactor;
51    address beneficiary;
52    address collateral_asset;
53    uint128 collateral_amount;
54    uint128 usde_amount;
55  }
```

EthenaMinting.sol, lines 36-38

```
36  bytes32 private constant ORDER_TYPE = keccak256(
37    "Order(string order_id,uint8 order_type,uint128 expiry,uint120 nonce,address benefactor,address
       beneficiary,address collateral_asset,uint128 collateral_amount,uint128 usde_amount)"
38  );
```

Field	Solidity structure	Literal EIP-712 type	Official TypeScript client
expiry	uint120	uint128	uint128
nonce	uint128	uint120	uint120

`encodeOrder` combines the fixed `ORDER_TYPE` hash with the actual structure values. The type-string width does not itself impose a runtime nonce bound. A client deriving its signing type from the Solidity structure will compute a different digest and can produce an invalid signature.

Integration result

The TypeScript example uses the literal widths in `ts/constants.ts`, lines 3-15; the Python example obtains the digest from `hashOrder`. Both avoid the mismatch. Reviewed SDK commit: `6f3a20dfe07eb6ad6d37d2ef24d428483796aecb`. [R5]

Recommendation

Publish a canonical signing schema and test client/contract digest equality at normal and boundary values. Coordinate type-hash changes with clients and outstanding orders. The mismatch alone does not establish signature forgery or a production failure.

Conclusions & Recommendations

The verified deployments enforce local blacklist restrictions and benefactor-specific nonce consumption. The cross-chain observation concerns the consistency of separately administered controls. The two minting findings concern interface clarity and client compatibility. The available evidence supports Informational classification for all three.

Recommended priorities

1. State whether restrictions are intended to apply independently per network or consistently to a holder across deployments; document the operational mechanism that implements that choice.
2. Monitor restriction-state divergence and exposure using pinned, error-checked account intersections, with explicit treatment of indirect holdings and in-flight transfers.
3. Define the supported nonce range and add compatibility tests for consumed-state migration before changing bitmap derivation.
4. Maintain canonical signing examples and contract/client digest tests; coordinate any schema transition with outstanding orders.

Rating rationale

The cross-chain rating depends on the intended restriction boundary and demonstrated impact, not on whether a test passes or on a third-party decision. Zero direct balances at two snapshots limit observed exposure; they do not prove the absence of a policy vulnerability. A confirmed requirement for synchronized restrictions, together with a reachable violation and resulting harm, would justify reassessment. Material operational harm or unauthorized loss would require a higher-impact analysis.

For the minting observations, the reviewed example clients avoid both the high-bit nonce collision and the schema mismatch. Their behavior is evidence about those examples, not a guarantee that every production integration uses the same scheme.

Disclaimer

This report is an independent technical assessment of the identified code paths and snapshots. It does not imply an engagement, certification, or endorsement by Ethena or any other organization. Findings and state measurements apply to the reviewed material and may not describe later changes.

A bounded security review cannot guarantee the absence of vulnerabilities. Tests provide evidence for exercised properties only. The exclusions and simulation assumptions in Scope and Methodology limit the assurance provided. No claim of complete authenticated bridge replay or production exploitation is made.

The report is provided for technical information without a warranty of correctness or security. It is not investment or legal advice. Remediation should be validated against the documented invariants and deployed state before a finding is marked resolved.

References & Reproduction Notes

R1. Ethereum sUSDe. StakedUSDeV2 verified source. Restriction-role implementation and underlying adapter asset.

R2. Ethereum adapter. StakedUSDeOFTAdapter verified source. `contracts/susde/StakedUSDeOFTAdapter.sol`, lines 38-55; `contracts/usde/USDeOFTAdapter.sol`, lines 62-68.

R3. Mantle OFT. StakedUSDeOFT verified source. `contracts/susde/StakedUSDeOFT.sol`, lines 56-59, 70-94; `contracts/usde/USDeOFT.sol`, lines 64-70; `contracts/libs/RateLimiter.sol`, lines 44-78.

R4. Ethereum minter. EthenaMinting verified source. `contracts/EthenaMinting.sol`, lines 36-38, 475-523, 560-568, 604-606; `contracts/interfaces/IEthenaMinting.sol`, lines 45-55.

R5. Official minting client. TypeScript nonce generation and signing, typed-data constants, and Python example. The repository is linked from Ethena API documentation.

Credit failure-handling note

In the adapter excerpt, line 47 decodes the return data before line 50 checks `success`. An empty-data revert from `hasRole` therefore reverts during decoding; it does not reach the advertised redirect branch. Three focused branch-model tests confirmed this order. The verified underlying token's `hasRole` is a mapping read, and no reachable production failure was demonstrated. This is a defensive-code note, not an additional exploitable finding. Check call success and return-data validity before decoding, then implement the intended failure policy explicitly.

Reproduction boundary

Use the block numbers and hashes in Source & State Verification, matching explorer bundles and compiler settings, and the companion `Evidence-Summary.json`. For the account intersection, replay `RoleGranted` and `RoleRevoked` for `FULL_RESTRICTED_STAKER_ROLE`, verify each active account with `hasRole`, and query the same address's Mantle `balanceOf` and `blackList` at the paired snapshot. RPC errors must not be interpreted as zero balances.

Source excerpts retain original line numbers. Full tests and raw research records are retained separately; they are not embedded in this report. Chain-state verification used the 7 October 2026 evidence set. The decode-order check and editorial review were completed on 8 October 2026; no new chain-state snapshot is implied.



Volodymyr Stetsenko

Independent Security Researcher

Independent security research focused on smart contracts and decentralized protocols. The work combines manual source analysis, adversarial testing, and deployment verification to produce clear, evidence-based findings and practical recommendations.
This report presents independently conducted research with explicit scope, documented observations, and measured impact assessments.

GitHub / VolodymyrStetsenko

LinkedIn / Volodymyr Stetsenko

X / volodymyr_sec

Independent Security Research Report

© 2026 Volodymyr Stetsenko. All rights reserved.